

JavaScript String Interview Questions

Part 2 — Advanced Practice Set (Q / Input / Code / Output)

25 Questions • Easy to Hard • With Explanations

Table of Contents

- Q1. Reverse words in a sentence
- Q2. Longest common prefix
- Q3. Check string rotation
- Q4. Remove duplicate characters
- Q5. Count words in a sentence
- Q6. Valid parentheses / brackets
- Q7. Longest substring without repeating chars
- Q8. Group anagrams
- Q9. All permutations of a string
- Q10. Caesar cipher encode/decode
- Q11. Run-length encoding
- Q12. Run-length decoding
- Q13. Roman numeral to integer
- Q14. Integer to Roman numeral
- Q15. Valid palindrome (alphanumeric only)
- Q16. Longest palindromic substring
- Q17. String compression
- Q18. Isomorphic strings
- Q19. Word break problem
- Q20. Multiply two numbers as strings
- Q21. Find all substrings of a string
- Q22. Letter case permutation
- Q23. Minimum window substring
- Q24. String to integer (atoi)
- Q25. Check if two strings are one edit apart

Interview Questions

Q1. Reverse the order of words in a sentence. Easy

Input:

```
let str = "Learn JavaScript Every Day";
```

Code:

```
function reverseWords(str) {
  return str.split(" ").reverse().join(" ");
}
console.log(reverseWords(str));
```

Output:
Day Every JavaScript Learn

Q2. Find the longest common prefix among an array of strings. Medium

Input:

```
let arr = ["flower", "flow", "flight"];
```

Code:

```
function longestCommonPrefix(arr) {
  if (arr.length === 0) return "";
  let prefix = arr[0];
  for (let i = 1; i < arr.length; i++) {
    while (arr[i].indexOf(prefix) !== 0) {
      prefix = prefix.slice(0, -1);
      if (!prefix) return "";
    }
  }
  return prefix;
}
console.log(longestCommonPrefix(arr));
```

Output:
fl

Q3. Check if one string is a rotation of another. Medium

Input:

```
let str1 = "waterbottle";
let str2 = "erbottlewat";
```

Code:

```
function isRotation(s1, s2) {
  return s1.length === s2.length && (s1 + s1).includes(s2);
}
console.log(isRotation(str1, str2));
```

Output:
true

Q4. Remove duplicate characters from a string, keeping first occurrence order. Easy

Input:

```
let str = "programming";
```

Code:

```
function removeDuplicates(str) {
  return [...new Set(str)].join("");
}
console.log(removeDuplicates(str));
```

Output:
progamin

Q5. Count the number of words in a sentence. Easy

Input:

```
let str = "  The quick  brown fox  ";
```

Code:

```
function countWords(str) {
```

```
    return str.trim().split(/\s+/).length;
  }
  console.log(countWords(str));
```

Output:
4

Q6. Check whether brackets/parentheses in a string are balanced.

Medium

Input:

```
let str1 = "{[() ()}";
let str2 = "{[()]}"
```

Code:

```
function isValid(str) {
  const pairs = { ")": "(", "]"": "["", "{"": "{" ";
  const stack = [];
  for (let ch of str) {
    if ([ "(", "[", "{" ].includes(ch)) stack.push(ch);
    else if (stack.pop() !== pairs[ch]) return false;
  }
  return stack.length === 0;
}
console.log(isValid(str1));
console.log(isValid(str2));
```

Output:
true
false

Q7. Find the length of the longest substring without repeating characters.

Medium

Input:

```
let str = "abcabcbb";
```

Code:

```
function lengthOfLongestSubstring(str) {
  let seen = new Map();
  let start = 0, max = 0;
  for (let end = 0; end < str.length; end++) {
    const ch = str[end];
    if (seen.has(ch) && seen.get(ch) >= start) {
      start = seen.get(ch) + 1;
    }
    seen.set(ch, end);
    max = Math.max(max, end - start + 1);
  }
  return max;
}
console.log(lengthOfLongestSubstring(str));
```

Output:
3 (substring "abc")

Q8. Group an array of strings into anagram groups.

Medium

Input:

```
let arr = ["eat", "tea", "tan", "ate", "nat", "bat"];
```

Code:

```
function groupAnagrams(arr) {
  const map = new Map();
  for (let word of arr) {
    const key = word.split("").sort().join("");
    if (!map.has(key)) map.set(key, []);
    map.get(key).push(word);
  }
  return [...map.values()];
}
console.log(groupAnagrams(arr));
```

Output:
[['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]

Q9. Generate all permutations of a string.

Medium

Input:

```
let str = "abc";
```

Code:

```
function permute(str) {
  if (str.length <= 1) return [str];
  let result = [];
  for (let i = 0; i < str.length; i++) {
    const rest = str.slice(0, i) + str.slice(i + 1);
    for (let perm of permute(rest)) {
      result.push(str[i] + perm);
    }
  }
  return result;
}
console.log(permute(str));
```

Output:
['abc', 'acb', 'bac', 'bca', 'cab', 'cba']

Q10. Implement a Caesar cipher (encode and decode). Medium

Input:

```
let str = "Hello";
let shift = 3;
```

Code:

```
function caesarEncode(str, shift) {
  return str.replace(/[a-zA-Z]/g, ch => {
    const base = ch === ch.toUpperCase() ? 65 : 97;
    return String.fromCharCode((ch.charCodeAt(0) - base + shift) % 26 + base);
  });
}
console.log(caesarEncode(str, shift));
```

Output:
Khoor

Q11. Run-length encode a string. Easy

Input:

```
let str = "aaabbbccd";
```

Code:

```
function encode(str) {
  let result = "";
  let count = 1;
  for (let i = 1; i <= str.length; i++) {
    if (str[i] === str[i - 1]) {
      count++;
    } else {
      result += str[i - 1] + count;
      count = 1;
    }
  }
  return result;
}
console.log(encode(str));
```

Output:
a3b3c2d1

Q12. Decode a run-length encoded string. Easy

Input:

```
let str = "a3b3c2d1";
```

Code:

```
function decode(str) {
  let result = "";
  const matches = str.match(/[a-zA-Z]\d+/g);
  for (let m of matches) {
    result += m[0].repeat(Number(m.slice(1)));
  }
  return result;
}
console.log(decode(str));
```

Output:
aaabbbccd

Q13. Convert a Roman numeral string to an integer. Medium

Input:

```
let str = "MCMXCIV";
```

Code:

```
function romanToInt(str) {
  const map = { I: 1, V: 5, X: 10, L: 50, C: 100, D: 500, M: 1000 };
  let total = 0;
  for (let i = 0; i < str.length; i++) {
    const curr = map[str[i]];
    const next = map[str[i + 1]];
    total += next > curr ? -curr : curr;
  }
  return total;
}
console.log(romanToInt(str));
```

Output:
1994

Q14. Convert an integer to a Roman numeral string.**Medium****Input:**

```
let num = 1994;
```

Code:

```
function intToRoman(num) {
  const values = [1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1];
  const symbols = ["M", "CM", "D", "CD", "C", "XC", "L", "XL", "X", "IX", "V", "IV", "I"];
  let result = "";
  for (let i = 0; i < values.length; i++) {
    while (num >= values[i]) {
      result += symbols[i];
      num -= values[i];
    }
  }
  return result;
}
console.log(intToRoman(num));
```

Output:
MCMXCIV

Q15. Check if a string is a valid palindrome, ignoring non-alphanumeric characters and case.**Medium****Input:**

```
let str = "A man, a plan, a canal: Panama";
```

Code:

```
function isValidPalindrome(str) {
  const clean = str.toLowerCase().replace(/[^a-z0-9]/g, "");
  return clean === clean.split("").reverse().join("");
}
console.log(isValidPalindrome(str));
```

Output:
true

Q16. Find the longest palindromic substring.**Hard****Input:**

```
let str = "babad";
```

Code:

```
function longestPalindrome(str) {
  let start = 0, maxLen = 1;
  function expand(l, r) {
    while (l >= 0 && r < str.length && str[l] === str[r]) { l--; r++; }
    if (r - l - 1 > maxLen) { start = l + 1; maxLen = r - l - 1; }
  }
  for (let i = 0; i < str.length; i++) {
    expand(i, i);
    expand(i, i + 1);
  }
  return str.substring(start, start + maxLen);
}
console.log(longestPalindrome(str));
```

Output:
bab (or "aba" — both valid)

Q17. Compress a string using counts of repeated characters (return original if not shorter).

Medium

Input:

```
let str = "aabcccccaaa";
```

Code:

```
function compress(str) {
  let result = "";
  let count = 1;
  for (let i = 1; i <= str.length; i++) {
    if (str[i] === str[i - 1]) {
      count++;
    } else {
      result += str[i - 1] + (count > 1 ? count : "");
      count = 1;
    }
  }
  return result.length < str.length ? result : str;
}
console.log(compress(str));
```

Output:
a2bc5a3

Q18. Check if two strings are isomorphic (characters map one-to-one).

Medium

Input:

```
let str1 = "egg";
let str2 = "add";
```

Code:

```
function isIsomorphic(s1, s2) {
  if (s1.length !== s2.length) return false;
  const map1 = {}, map2 = {};
  for (let i = 0; i < s1.length; i++) {
    const a = s1[i], b = s2[i];
    if (map1[a] && map1[a] !== b) return false;
    if (map2[b] && map2[b] !== a) return false;
    map1[a] = b;
    map2[b] = a;
  }
  return true;
}
console.log(isIsomorphic(str1, str2));
```

Output:
true

Q19. Word Break: check if a string can be segmented into dictionary words.

Hard

Input:

```
let str = "leetcode";
let dict = ["leet", "code"];
```

Code:

```
function wordBreak(str, dict) {
  const wordSet = new Set(dict);
  const dp = Array(str.length + 1).fill(false);
  dp[0] = true;
  for (let i = 1; i <= str.length; i++) {
    for (let j = 0; j < i; j++) {
      if (dp[j] && wordSet.has(str.slice(j, i))) {
        dp[i] = true;
        break;
      }
    }
  }
  return dp[str.length];
}
console.log(wordBreak(str, dict));
```

Output:
true

Q20. Multiply two non-negative numbers represented as strings.

Hard

Input:

```
let num1 = "123";
let num2 = "456";
```

Code:

```
function multiply(num1, num2) {
  if (num1 === "0" || num2 === "0") return "0";
  const result = new Array(num1.length + num2.length).fill(0);
  for (let i = num1.length - 1; i >= 0; i--) {
    for (let j = num2.length - 1; j >= 0; j--) {
      const mul = (num1[i] - "0") * (num2[j] - "0");
      const sum = mul + result[i + j + 1];
      result[i + j + 1] = sum % 10;
      result[i + j] += Math.floor(sum / 10);
    }
  }
  return result.join("").replace(/^(0+)/, "");
}
console.log(multiply(num1, num2));
```

Output:
56088

Q21. Find all substrings of a given string.

Easy

Input:

```
let str = "abc";
```

Code:

```
function allSubstrings(str) {
  let result = [];
  for (let i = 0; i < str.length; i++) {
    for (let j = i + 1; j <= str.length; j++) {
      result.push(str.slice(i, j));
    }
  }
  return result;
}
console.log(allSubstrings(str));
```

Output:
['a', 'ab', 'abc', 'b', 'bc', 'c']

Q22. Generate all letter-case permutations of a string containing letters and digits.

Medium

Input:

```
let str = "a1b";
```

Code:

```
function letterCasePermutation(str) {
  let result = [""];
  for (let ch of str) {
    if (/[a-zA-Z]/.test(ch)) {
      result = result.flatMap(p => [p + ch.toLowerCase(), p + ch.toUpperCase()]);
    } else {
      result = result.map(p => p + ch);
    }
  }
  return result;
}
console.log(letterCasePermutation(str));
```

Output:
['a1b', 'a1B', 'A1b', 'A1B']

Q23. Find the minimum window substring of s that contains all characters of t.

Hard

Input:

```
let s = "ADOBECODEBANC";
let t = "ABC";
```

Code:

```
function minWindow(s, t) {
  const need = {};
  for (let ch of t) need[ch] = (need[ch] || 0) + 1;
  let required = Object.keys(need).length;
  let left = 0, formed = 0;
  const window = {};
  let best = [Infinity, 0, 0];

  for (let right = 0; right < s.length; right++) {
    const ch = s[right];
    window[ch] = (window[ch] || 0) + 1;
    if (need[ch] && window[ch] === need[ch]) formed++;

    while (formed === required) {
      if (right - left + 1 < best[0]) best = [right - left + 1, left, right];
      const lch = s[left];

```

```

        window[lch]--;
        if (need[lch] && window[lch] < need[lch]) formed--;
        left++;
    }
}
return best[0] === Infinity ? "" : s.substring(best[1], best[2] + 1);
}
console.log(minWindow(s, t));

```

Output:
BANC

Q24. Implement atoi: convert a string to an integer (myAtoi).

Medium

Input:

```
let str = " -42abc";
```

Code:

```

function myAtoi(str) {
    const match = str.trim().match(/^[\+]?[0-9]+/);
    if (!match) return 0;
    let num = parseInt(match[0], 10);
    const INT_MAX = 2 ** 31 - 1, INT_MIN = -(2 ** 31);
    return Math.max(INT_MIN, Math.min(INT_MAX, num));
}
console.log(myAtoi(str));

```

Output:
-42

Q25. Check if two strings are exactly one edit (insert, delete, replace) apart.

Medium

Input:

```
let str1 = "pale";
let str2 = "ple";
```

Code:

```

function isOneEditAway(s1, s2) {
    if (Math.abs(s1.length - s2.length) > 1) return false;
    const [shorter, longer] = s1.length < s2.length ? [s1, s2] : [s2, s1];
    let i = 0, j = 0, edited = false;
    while (i < shorter.length && j < longer.length) {
        if (shorter[i] !== longer[j]) {
            if (edited) return false;
            edited = true;
            if (shorter.length === longer.length) i++;
        } else {
            i++;
        }
        j++;
    }
    return true;
}
console.log(isOneEditAway(str1, str2));

```

Output:
true